FROM THEORY TO CODE · DJANGO REST FRAMEWORK

Registration Search API

A teacher-led walkthrough of the CRM code and a live Postman demonstration.

Name and email search · DOB filters · Ordering · Pagination · JWT

What will students be able to do?

Lesson step 1 of 25

Show: The existing `/account/api/registrations/` list can now search names and emails, filter by an exact email or date-of-birth range, order results, and return one page at a time. The same API still requires a JWT access token.

> **TEACHER EXPLANATION**
>
> This is a practical DRF lesson. We are extending the Registration API students already built; we are not creating a separate search service. By the end, students should be able to read a URL such as `?search=testcase&dob_from=2026-09-01&dob_to=2026-09-30`, predict which CRM customers should remain, and explain the JSON response.

Opening question: If a teacher remembers a customer's name but not their registration ID, how should the CRM find the customer?

The CRM pieces used today

Lesson step 2 of 25

Show:

Piece	What it holds or does
`Registration`	`id`, `name`, `email`, `password`, `date_of_birth`
`RegistrationSerializer`	Controls fields accepted and returned by the Registration API
`RegistrationViewSet`	Provides the list and detail API actions
Existing route	`GET /account/api/registrations/`
Django User	Logs into the JWT API; separate from a `Registration` customer

TEACHER EXPLANATION

The customer list in the HTML page and this API are separate views. The HTML customer's search box is still only visual. The API work in this lesson does not connect that box. The Bill model is also outside this implementation lesson because it does not yet have an API.

Why extend the list API?

Show: Before this change, `GET /account/api/registrations/` returned the list, but adding `?search=Teacher` did not make the ViewSet search. Knowing an ID allowed a detail lookup; it did not help someone who knew only a name or email.

```
GET /account/api/registrations/2/           -> customer with ID 2
GET /account/api/registrations/?search=Teacher -> customers matching Teacher
```

TEACHER EXPLANATION

Both requests use the route that already exists. The first selects one known ID. The second searches the collection using a query parameter. Search happens in Django/database code before the response is serialized and sent to the client. This can reduce how many rows the client receives. DRF's default partial search is not automatically fast on a large database; indexes and query measurement would be separate scaling work.

Read a search URL

Show:

```
GET /account/api/registrations/?search=Teacher
    └──────────── list endpoint ──────────┘ └ query parameter ┘
```

> **TEACHER EXPLANATION**
>
> `GET` means read data. The path chooses the Registration list. The `?` starts the query string, and `search=Teacher` supplies a value to DRF's `SearchFilter`. Multiple parameters are joined with `&`. The route did not change when we added filtering. A valid JWT is still required.

Ask students: Would the API need a new URL for `?email=teacher.test@example.com`? Answer: no; it is another parameter on the same list route.

What happens inside the request?

Lesson step 5 of 25

Show:

```
GET URL + Bearer token
  -> JWT authentication checks the token
  -> get_queryset() applies exact email and DOB bounds
  -> SearchFilter applies ?search=
  -> OrderingFilter chooses the order
  -> pagination selects the requested page
  -> RegistrationSerializer builds the JSON without password
```

> **TEACHER EXPLANATION**
>
> The features work together. A customer must pass each requested structured filter and the text search. Ordering arranges those matches, and pagination sends only one part of them. The serializer controls what each returned customer looks like. If authentication fails, the request stops with `401` before customers are returned.

Which code files changed?

Show:

File	Change
`crm/registration/serializers.py`	Hide `password` from API responses
`crm/registration/views.py`	Add search, exact email and DOB filters, ordering, and pagination

TEACHER EXPLANATION

The router already supplied `/account/api/registrations/`. We did not add a URL, model field, migration, or package for this lesson. Students should look for each new behavior in the existing serializer and ViewSet.

Serializer: accept password input, omit it from output

Lesson step 7 of 25

Show the code in `RegistrationSerializer.Meta`:

```
fields = ['id', 'name', 'email', 'password', 'date_of_birth']
extra_kwargs = {
    'password': {'write_only': True},
}
```

TEACHER EXPLANATION

The same `ModelViewSet` can accept create/update requests and return customer records. `write_only` means a password may be sent in an API request but is not included when the serializer produces a response. In the live list, each record had `id`, `name`, `email`, and `date_of_birth`; no record had a `password` key. This setting only controls serialization. The custom `Registration` model still stores its password as plain text, which requires separate work before real production use.

Memory phrase: It may go **in**; it does not come **out**.

Text search with DRF `SearchFilter`

Show the code in `RegistrationViewSet`:

```
filter_backends = [SearchFilter, OrderingFilter]
search_fields = ['name', 'email']
```

> **TEACHER EXPLANATION**
>
> `SearchFilter` reads the `search` query parameter. `search_fields` allows it to check only the Registration `name` and `email` fields. With the current DRF default, matching is case-insensitive and partial. A record may match in either field. A password is never a searchable field. If `SearchFilter` were absent, `?search=Teacher` would not change the list.

Live example: `GET /account/api/registrations/?search=Teacher` returned one result: `Teacher Test`, ID 2. A lowercase `teacher` would match it too.

Exact email is a different rule

Lesson step 9 of 25

Show the first part of `get_queryset()`:

```
def get_queryset(self):
    queryset = super().get_queryset()
    email = self.request.query_params.get('email')

    if email:
        queryset = queryset.filter(email__iexact=email)
```

TEACHER EXPLANATION

`super().get_queryset()` starts with the Registration records configured on the ViewSet. `query_params.get('email')` reads `?email=` from the URL. If it was supplied, `email__iexact` keeps only a whole email match while ignoring letter case. This is useful when the caller knows a particular customer's full email. If no email parameter is supplied, this part leaves the queryset alone.

Request	Live result	Why
`?email=teacher.test@example.com`	`count: 1`	Full email matches
`?email=teacher.test`	`count: 0`	A fragment is not a full email
`?search=teacher.test`	Matches Teacher Test	General search accepts a fragment

Date filters read text from the URL

Show:

```
from_text = self.request.query_params.get('dob_from')
to_text = self.request.query_params.get('dob_to')

try:
    from_date = parse_date(from_text) if from_text else None
    to_date = parse_date(to_text) if to_text else None
except ValueError:
    raise ValidationError({'date': 'Use YYYY-MM-DD for dob_from and dob_to.'})
```

TEACHER EXPLANATION

A URL contains text, even when that text looks like a date. `parse_date()` turns the supplied text into date values that Django can compare. Either bound is optional. If the supplied text cannot represent a valid calendar date, the API returns a `400` error with a useful message. These checks validate the **search input**; they do not validate a customer's saved date of birth when a customer is created.

Example: `dob_from=banana` is an invalid filter value. A saved customer DOB such as `2027-02-01` is a different piece of data; this search code does not decide whether it is a realistic DOB.

Date range errors and inclusive bounds

Lesson step 11 of 25

Show the remaining date logic:

```
if (from_text and from_date is None) or (to_text and to_date is None):
    raise ValidationError({'date': 'Use YYYY-MM-DD for dob_from and dob_to.'})

if from_date and to_date and from_date > to_date:
    raise ValidationError({'date': 'dob_from must be before or equal to dob_to.'})

if from_date:
    queryset = queryset.filter(date_of_birth__gte=from_date)
if to_date:
    queryset = queryset.filter(date_of_birth__lte=to_date)

return queryset
```

TEACHER EXPLANATION

The first check catches date text the parser could not understand. The second rejects a start after the end. `gte` means on or after; `lte` means on or before. The boundaries themselves are included. A customer with a `null` DOB does not match a date range. The final `return queryset` hands the narrowed records back to DRF.

Live example: `?dob_from=2026-09-01&dob_to=2026-09-30` returned three records, including customers born on September 1 and September 30.

Combine search and filters

Show:

```
GET /account/api/registrations/?search=testcase&dob_from=2026-09-01&dob_to=2026-09-30
```

> **TEACHER EXPLANATION**
>
> Text search can match a term in `name` **or** `email`. The result must **also** satisfy the date range. In the live data, the search term `testcase` matched multiple names, but only `testcase15` had a DOB inside this September 2026 range. The final response had `count: 1` and ID 14. A request with `search=Teacher` and the same date range returned `count: 0` because Teacher Test has a `null` DOB.

Student prediction: Before sending the request, ask which customer should survive both conditions.

Ordering the matches

Lesson step 13 of 25

Show:

```
filter_backends = [SearchFilter, OrderingFilter]
ordering = ['id']
ordering_fields = ['id', 'name', 'email', 'date_of_birth']
```

> **TEACHER EXPLANATION**
>
> `OrderingFilter` reads `?ordering=`. The `ordering_fields` list limits the fields a client may use. If no ordering parameter is sent, the ViewSet orders by ID. A leading minus sign means descending order. Ordering matters especially with pagination: page 1 and page 2 need a predictable sequence. For the classroom demo, `-id` avoids ambiguity from records with missing DOBs.

Live example: `?ordering=-id&page_size=3&page=1` showed IDs `14`, `13`, and `10` first. Without the minus sign, lower IDs come first.

Pagination: one page of results

Lesson step 14 of 25

Show:

```
class RegistrationPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 50

class RegistrationViewSet(viewsets.ModelViewSet):
    pagination_class = RegistrationPagination
```

TEACHER EXPLANATION

The default page holds ten records. A client may request a different page size with `page_size`, but no more than 50. `page` selects which page to read. `pagination_class` connects these rules to this ViewSet. Pagination applies after filtering and ordering, so `count` is the total number of matches, not just the number on the current page.

Live example: With nine matching customers, `?page_size=3&page=2` gives `count: 9`, three items in `results`, and non-null `previous` and `next` links.

Read the JSON response

Lesson step 15 of 25

Show a live-result example for `?search=Teacher`:

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 2,
      "name": "Teacher Test",
      "email": "teacher.test@example.com",
      "date_of_birth": null
    }
  ]
}
```

TEACHER EXPLANATION

`count` is the total number of records left after search and filters. `results` contains the current page. `next` and `previous` are page links or `null` when there is no such page. The password is absent because the serializer marks it write-only. A valid search with no matches returns `200 OK`, `count: 0`, and `results: []`; that is different from a bad request.

Start the live project

Show the terminal commands:

```
cd /data/2026/7_CRM_CLASS/batch_4_project
docker compose up -d db
docker compose ps
source .venv/bin/activate
cd crm
python manage.py migrate
python manage.py runserver 127.0.0.1:8001
```

> **TEACHER EXPLANATION**
>
> PostgreSQL runs in Docker; Django runs locally from the virtual environment. `docker compose ps` should show `crm-postgres` as healthy. `migrate` is safe to run again and may report no migrations to apply. Keep the Django terminal running while using Postman. The development server is at `http://127.0.0.1:8001/`.

Expected startup lines: `System check identified no issues` and `Starting WSGI development server at http://127.0.0.1:8001/`.

Postman request 1: get a JWT

Show the saved request `1_CRM login`:

```
POST http://127.0.0.1:8001/account/api/login/
Content-Type: application/json

{"email":"<your Django user email>","password":"<your password>"}
```

> **TEACHER EXPLANATION**
>
> Set Postman Body to **raw → JSON** and replace the placeholders with the Django User credentials you created with `createsuperuser`. This login uses a **Django User**, not a customer in `Registration`. The instructor's local demo credentials are recorded in the private `PROJECT_MEMORY.md` file. A successful login gives `200 OK` with an `access` and a `refresh` token. The access token is what the Registration API needs.

Possible response shape:

```
{"refresh":"<long refresh token>","access":"<long access token>"}
```

Postman saves the access token automatically

Lesson step 18 of 25

Show `1_CRM login` → Scripts → Post-response:

```
if (pm.response.code === 200) {
    pm.collectionVariables.set("access_token", pm.response.json().access);
}
```

TEACHER EXPLANATION

This runs after the login response arrives. If the response is successful, Postman reads the `access` field and saves it in a collection variable named `access_token`. Send `1_CRM login` after saving the script. If the login request has not been sent since the script was added, the variable may still be empty. This script is Postman test/demo setup, not Django server code.

Postman request 2: use the token

Lesson step 19 of 25

Show the saved request `2_CRM Registration Search`:

```
GET http://127.0.0.1:8001/account/api/registrations/
Authorization: Bearer <access token>
```

Postman setting: Authorization → Auth Type **Bearer Token** → Token field `{{access_token}}`. Do not type `Bearer` into Postman's Token field; Postman adds that word to the header.

> **TEACHER EXPLANATION**
>
> The double braces mean “insert the collection variable saved by request 1.” Send login first, then send this GET request. A valid token gives `200 OK` and the paginated list. A missing or expired token gives `401 Unauthorized`; send login again to get a new access token. A browser address bar alone cannot add this Bearer header to the request.

Postman Params: turn one feature on at a time

Lesson step 20 of 25

Show: Postman **Params** rows become query parameters in the URL. A checked row is sent; an unchecked row is not sent. The URL at the top is the best way to confirm what Postman will send.

Checked Params row	Request	Live result
`search = Teacher`	`?search=Teacher`	`count: 1`, Teacher Test
`email = teacher.test@example.com`	`?email=teacher.test@example.com`	`count: 1`
`email = teacher.test`	`?email=teacher.test`	`count: 0`

TEACHER EXPLANATION

The first is a partial text search over name or email. The latter two use the exact email filter. Show the URL changing when a checkbox is toggled. In the live lesson, an old checked `search=Teacher` combined with a date range produced zero records; that was correct because Teacher Test has no DOB in that range. Students should always read the full URL before debugging the backend.

Postman Params: dates and combined conditions

Lesson step 21 of 25

Show these requests and expected responses:

Request suffix	Live result
`?dob_from=2026-09-01`	`count: 4`; includes a later 2027 DOB because no upper bound was sent
`?dob_from=2026-09-01&dob_to=2026-09-30`	`count: 3`
`?search=testcase&dob_from=2026-09-01&dob_to=2026-09-30`	`count: 1`, testcase15, ID 14

TEACHER EXPLANATION

In Postman, check both date rows to send a range. If `dob_to` is unchecked, the API receives only the lower bound. The combined request illustrates the rule: a returned customer must match the text search and fall inside the date range. If a student's count differs, first inspect the checked Params rows and the complete URL.

Postman Params: pages and order

Lesson step 22 of 25

Show:

Request suffix	Expected live result
`?page_size=3&page=1`	Three records in `results`; `count: 9`; `next` is a link
`?page_size=3&page=2`	The next three records; `count` remains 9
`?ordering=-id&page_size=3&page=1`	First IDs `14`, `13`, `10`

TEACHER EXPLANATION

A page never changes the total match count; it changes which part appears in `results`. Ordering controls the sequence before the page is taken. For this nine-row data set, a page size of three visibly creates three pages without adding test customers.

Distinguish four important responses

Show:

Case	Example	HTTP status and body idea
Valid request, no match	`?email=teacher.test`	`200`, `count: 0`, `results: []`
Bad filter input	`?dob_from=not-a-date`	`400`, `{"date":"Use YYYY-MM-DD for dob_from and dob_to."}`
Start after end	`?dob_from=2026-10-01&dob_to=2026-09-30`	`400`, `{"date":"dob_from must be before or equal to dob_to."}`
No Bearer token	Plain GET of the list	`401`, authentication credentials not provided

TEACHER EXPLANATION

A zero result is not an error; it is a successful search with no matches. An invalid filter is a client mistake, so the API explains it with `400`. A missing token is an authentication problem, so the API returns `401`. These distinctions help a frontend show the right message to a user.

Live teaching sequence

Lesson step 24 of 25

Show the order to run in class:

1. Start Docker PostgreSQL and Django. Keep the server terminal open.
2. Send `1_CRM login` in Postman; point out `200`, `access`, and the Post-response script.
3. Send `2_CRM Registration Search` with no checked Params; point out `count`, `results`, and the absence of `password`.
4. Check `search=Teacher`, then try the full and partial email values.
5. Check both September 2026 DOB bounds, then combine with `search=testcase`.
6. Use `page_size=3`, change `page`, and use `ordering=-id`.
7. Show one `400` invalid-date response and one `401` request without a valid token.

> **TEACHER EXPLANATION**
>
> Before every Send, ask students to predict the status code, count, and first matching record. Then compare the prediction with the JSON. Keep the checked Params rows tidy; duplicated `email` or date rows can make the demonstration confusing. When finished, uncheck all Params and save the base request.

Current scope and next steps

Lesson step 25 of 25

Show: The lesson provides a basic server-side Registration search API with JWT access, allowed search/order fields, exact and range filters, clear errors, safe response fields, and pagination. It is a useful foundation for a product API.

> **TEACHER EXPLANATION**
>
> The current `SearchFilter` uses simple partial matching. It does not rank the best match, fix spelling mistakes, or automatically make large searches fast. The HTML customer search box is still not connected to this API. The Bill API is not implemented. `write_only` keeps Registration passwords out of API responses, but the Registration model still stores them as plain text. These are future improvements, not features to claim as complete today.

Final student question: Which part of today's code would you change to search another safe text field? Which part would you change to permit a new ordering field?

Code reference: serializer and imports

Teacher reference

`crm/registration/serializers.py`:

```
class RegistrationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Registration
        fields = ['id', 'name', 'email', 'password', 'date_of_birth']
        extra_kwargs = {'password': {'write_only': True}}
```

Additional imports in `crm/registration/views.py`:

```
from rest_framework.filters import SearchFilter, OrderingFilter
from rest_framework.pagination import PageNumberPagination
from django.utils.dateparse import parse_date
from rest_framework.exceptions import ValidationError
```

Code reference: ViewSet setup

Teacher reference · matches the project

```
class RegistrationPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 50


class RegistrationViewSet(viewsets.ModelViewSet):
    queryset = Registration.objects.all()
    serializer_class = RegistrationSerializer
    permission_classes = [IsAuthenticated]
    pagination_class = RegistrationPagination

    filter_backends = [SearchFilter, OrderingFilter]
    search_fields = ['name', 'email']
    ordering = ['id']
    ordering_fields = ['id', 'name', 'email', 'date_of_birth']
```

Code reference: exact email and DOB filters

Teacher reference · matches the project

```
    def get_queryset(self):
        queryset = super().get_queryset()

        email = self.request.query_params.get('email')
        if email:
            queryset = queryset.filter(email__iexact=email)

        from_text = self.request.query_params.get('dob_from')
        to_text = self.request.query_params.get('dob_to')

        try:
            from_date = parse_date(from_text) if from_text else None
            to_date = parse_date(to_text) if to_text else None
        except ValueError:
            raise ValidationError({'date': 'Use YYYY-MM-DD for dob_from and dob_to.'})

        if (from_text and from_date is None) or (to_text and to_date is None):
            raise ValidationError({'date': 'Use YYYY-MM-DD for dob_from and dob_to.'})

        if from_date and to_date and from_date > to_date:
            raise ValidationError({'date': 'dob_from must be before or equal to dob_to.'})

        if from_date:
            queryset = queryset.filter(date_of_birth__gte=from_date)
        if to_date:
            queryset = queryset.filter(date_of_birth__lte=to_date)

        return queryset
```

Postman request cheat sheet · 1

Teacher reference · exact request URLs

Every Registration GET uses Bearer Token `{{access_token}}`. Send login first.

Purpose	Method and URL	Expected live result
Full list	`GET http://127.0.0.1:8001/account/api/registrations/`	`200`, count 9; password absent
Partial text	`GET http://127.0.0.1:8001/account/api/registrations/?search=Teacher`	`200`, Teacher Test
Exact email	`GET http://127.0.0.1:8001/account/api/registrations/?email=teacher.test@example.com`	`200`, count 1
Date range	`GET http://127.0.0.1:8001/account/api/registrations/?dob_from=2026-09-01&dob_to=2026-09-30`	`200`, count 3

Postman request cheat sheet · 2

Teacher reference · exact request URLs

Every Registration GET uses Bearer Token `{{access_token}}`. Send login first.

Purpose	Method and URL	Expected live result
Combined	`GET http://127.0.0.1:8001/account/api/registrations/?search=testcase&dob_from=2026-09-01&dob_to=2026-09-30`	`200`, testcase15
Paged	`GET http://127.0.0.1:8001/account/api/registrations/?page_size=3&page=2`	`200`, three results from page 2
Descending	`GET http://127.0.0.1:8001/account/api/registrations/?ordering=-id&page_size=3&page=1`	`200`, IDs 14, 13, 10
Bad date	`GET http://127.0.0.1:8001/account/api/registrations/?dob_from=not-a-date`	`400`, date explanation

Sources and course note

Teacher reference

- DRF filtering: `SearchFilter` and `OrderingFilter`
- DRF `PageNumberPagination`
- Postman: Params
- Postman: Post-response scripts
- Postman: collection variables